

LangGraph: Understanding State

The core shared memory architecture for multi-actor LLM applications

What is State?

In LangGraph, **State** is the central, shared memory layer that connects all nodes in your graph. Unlike standard LCEL chains where data merely passes sequentially from one step to the next, LangGraph agents revolve around a persistent state object. Think of it as a shared whiteboard: each node (actor) reads from the board, performs its execution, and writes its updates back.

How Nodes Interact with State

Nodes are defined as standard Python functions. A node receives the current state as input, but importantly, **it does not mutate the state directly**. Instead, a node returns a dictionary of *partial updates*. LangGraph's engine takes these updates and applies them to the global state object before routing to the next node.

```
[Current State] → (Node A Executing) →  
Returns {"key": "update"}  
↓  
[LangGraph Reducer Merges Update]  
↓  
[Updated State] → (Routes to Node B)
```

State Schemas: TypedDict

State is typically defined using Python's TypedDict (or Pydantic models). This acts as a formal contract between nodes, offering IDE type-checking without runtime overhead. When you initialize a StateGraph, you pass this schema to define the allowed fields.

Example: Defining a Stateful Schema

The following example demonstrates a schema where the messages field accumulates (using a reducer) while the topic field simply overwrites:

```
import operator
```

Reducers: The Merge Logic

When a node returns an update for a field, LangGraph needs to know how to combine the new value with the old one. This is handled by a **Reducer**.

- **Default (Overwrite):** If no reducer is specified, standard fields use a "last-write-wins" approach. Returning a new value completely replaces the old one.
- **Custom Reducers (Accumulate):** For fields like chat history, overwriting is destructive. By using `typing.Annotated`, you can attach a reducer function.

Using Annotated and Built-ins

You can use standard Python operators like `operator.add` to append to lists. LangGraph also provides specialized reducers like `add_messages` to handle complex message appending and updating seamlessly.

```

from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END

# Define the State Schema
class AgentState(TypedDict):
    # 'Annotated' tells LangGraph to use operator.add to append/merge lists
    # rather than replacing the existing list of messages.
    messages: Annotated[list, operator.add]
    topic: str # Plain field: last-write-wins (Overwritten by updates)

# Define a Node
def chatbot_node(state: AgentState):
    # The node reads the current state
    current_topic = state.get("topic", "General")

    # It returns ONLY the partial updates
    return {
        "messages": ["New response from assistant"], # This gets appended
        "topic": "Specific Task" # This overwrites
    }

# Compile Graph
graph = StateGraph(AgentState)
graph.add_node("chatbot", chatbot_node)

```

Best Practices & Common Pitfalls

- **Do not mutate state in place:** Nodes should be pure functions. Never use `state["messages"].append()` inside a node. Always return the new items in a dictionary.
- **Return only what changed:** You do not need to return the entire state object. Return only the fields you intend to update.
- **Parallel Execution Risks:** If two nodes run in parallel and write to the same plain field (without a reducer), LangGraph will throw an `InvalidUpdateError` to prevent silent overwrites.
- **Initialize Empty Collections:** Reducers must be able to handle an empty initial state.